

Name: Kallepalli Sai Meghana

SRN: PES1UG24AM911

Section: E

CLAUDE CHAT HISTORY : <https://claude.ai/chat/1963e9e3-f8ff-477f-a6c8-d3202a98da2f>

Tic-Tac-Toe Lab: LLM Chat Transcript

Repository: https://github.com/SETAPESU26/09_tic_tac_toe

LLM used: Claude (Anthropic), claude.ai

Date: 6 October 2026

Exchange 1. Task 1: Win and draw detection, banner fix

Prompt

I'm doing a lab where I fix a Pygame Tic-Tac-Toe game with an LLM's help. Repo: https://github.com/SETAPESU26/09_tic_tac_toe

Read the README and every file in tic-tac-toe/ first. Then do Task 1 only:

1. diagonal wins aren't detected
2. a winning move that fills the last cell is reported as "Draw!"
3. clicks are still accepted after the round ends

Also, the result banner is drawn partly below the window and is too wide, so make it fully visible, and show "Round over" instead of the turn label once a round ends.

Don't fix the occupied-cell bug yet, that's a separate task. Give me full replacements of every file you change, explain each fix in a line, and test it before answering.

Response

I read the README and all seven files in tic-tac-toe/, made the changes below, and ran them headlessly before writing this.

What was wrong and the fix (one line each)

1. **Diagonals:** `check_winner` only built 6 lines (rows and columns), so I added the two diagonals to the list it checks.
2. **Win reported as "Draw!":** `check_round_end` checked "board full" before "winner", so I now check for a winner first and only call it a draw if there's no winner and the board is full.
3. **Clicks after the round ends:** `handle_click` never looked at `round_over`, so it now returns immediately once the round is over. The old code only blocked clicks by accident after an X win, because the turn had flipped to O. After an O win or a draw, clicks still went through.
4. **Banner off-screen:** it was centred at `y = 500`, the bottom edge of a 500 px window, so half of it was cut off. It also measured 416 px wide in a 400 px window. It's now centred in the 40 px strip below the board, with 10 px side margins. If the text doesn't fit at size 22, it steps the font down until it does (it lands on size 20), and it's clamped to the window.

5. **"Round over" label:** `GameEngine.draw` shows "Round over" instead of the turn label when `round_over` is true.

`main.py` changed by one line so the font name and size come from `renderer.py`. Otherwise, the shrink-to-fit code in the `renderer` would need its own copy of "consolas", 22. The occupied-cell overwrite in `handle_click` is untouched; its # BUG comment is still there for Task 3.

Full replacements

`tic-tac-toe/game/rules.py`

```
"""
rules: win/draw detection for a 3x3 board.

board is a 3x3 list of lists, each cell is None, 'X', or 'O'.
"""

def check_winner(board):
    lines = []
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])
    lines.append([board[0][0], board[1][1], board[2][2]]) # top-left to bottom-right
    lines.append([board[0][2], board[1][1], board[2][0]]) # top-right to bottom-left

    for line in lines:
        if line[0] is not None and line[0] == line[1] == line[2]:
            return line[0]
    return None

def is_board_full(board):
    return all(cell is not None for row in board for cell in row)
```

`tic-tac-toe/game/game_engine.py`

```
"""
GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py) - this is given infrastructure, not
something you need to build.

Starter version: no scoreboard yet, no first-player choice, and only
one combined reset control. Move validation has a known gap (see
handle_click) that Task 3 asks you to fix.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'

class GameEngine:
    def __init__(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = 'X'
        self.round_over = False
        self.winner = None # 'X', 'O', or None (meaning draw, only valid when round_over)

    def handle_click(self, pos):
        if self.round_over:
```

```

        return # the round has ended - ignore clicks until R starts a new one
    if self.current_player != HUMAN_SYMBOL:
        return # not your turn - the computer is about to move (or already has)
    cell = board_pos_to_cell(pos)
    if cell is None:
        return
    row, col = cell
    self.board[row][col] = self.current_player # BUG: doesn't check if the cell is already occupied
    self.check_round_end()
    self.current_player = 'O' if self.current_player == 'X' else 'X'
    self._maybe_take_computer_turn()

def _maybe_take_computer_turn(self):
    if self.round_over or self.current_player != COMPUTER_SYMBOL:
        return
    move = choose_move(self.board)
    if move is None:
        return
    row, col = move
    self.board[row][col] = self.current_player
    self.check_round_end()
    self.current_player = 'O' if self.current_player == 'X' else 'X'

def handle_keydown(self, key):
    import pygame
    if key == pygame.K_r:
        self.__init__()

def check_round_end(self):
    winner = check_winner(self.board) # a win takes priority over a full board
    if winner:
        self.round_over = True
        self.winner = winner
    elif is_board_full(self.board):
        self.round_over = True
        self.winner = None

def draw(self, surface, font):
    from game import renderer
    renderer.draw_board(surface, self.board)
    if self.round_over:
        turn_label = "Round over"
    elif self.current_player == HUMAN_SYMBOL:
        turn_label = "Your turn (X)"
    else:
        turn_label = "Computer's turn (O)"
    renderer.draw_text(surface, font, turn_label, (10, 20))

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"
        renderer.draw_banner(surface, font, f"{text} Press R for a new round.")

```

tic-tac-toe/game/renderer.py

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import functools

import pygame

WIDTH, HEIGHT = 400, 500
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
WINDOW_SIZE = (WIDTH, HEIGHT)

FONT_NAME = "consolas"
FONT_SIZE = 22
MIN_BANNER_FONT_SIZE = 12
BANNER_MARGIN = 10

```

```

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
COLOR_BANNER = (180, 40, 40)

@functools.lru_cache(maxsize=None)
def _font(size):
    return pygame.font.SysFont(FONT_NAME, size)

def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP
    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None
    col = x // CELL_SIZE
    row = y // CELL_SIZE
    return int(row), int(col)

def draw_board(surface, board):
    surface.fill(COLOR_BG)
    for i in range(1, 3):
        pygame.draw.line(surface, COLOR_LINE, (i * CELL_SIZE, BOARD_TOP), (i * CELL_SIZE, BOARD_TOP +
BOARD_SIZE), 3)
        pygame.draw.line(surface, COLOR_LINE, (0, BOARD_TOP + i * CELL_SIZE), (BOARD_SIZE, BOARD_TOP + i *
CELL_SIZE), 3)

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]
            if symbol is None:
                continue
            center = (c * CELL_SIZE + CELL_SIZE // 2, BOARD_TOP + r * CELL_SIZE + CELL_SIZE // 2)
            if symbol == 'X':
                offset = CELL_SIZE // 3
                pygame.draw.line(surface, COLOR_X, (center[0]-offset, center[1]-offset), (center[0]+offset,
center[1]+offset), 6)
                pygame.draw.line(surface, COLOR_X, (center[0]+offset, center[1]-offset), (center[0]-offset,
center[1]+offset), 6)
            else:
                pygame.draw.circle(surface, COLOR_O, center, CELL_SIZE // 3, 6)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    # The banner lives in the strip between the bottom of the board and
    # the bottom of the window, with a small margin on the left and right.
    strip_top = BOARD_TOP + BOARD_SIZE
    area = pygame.Rect(
        BANNER_MARGIN,
        strip_top,
        surface.get_width() - 2 * BANNER_MARGIN,
        surface.get_height() - strip_top,
    )

    # Use the normal font if the text fits; otherwise step the size down
    # until it does, so the banner is never cut off at the window edges.
    banner_font = font
    width, height = banner_font.size(text)
    size = FONT_SIZE
    while (width > area.width or height > area.height) and size > MIN_BANNER_FONT_SIZE:
        size -= 1
        banner_font = _font(size)
        width, height = banner_font.size(text)

```

```

surf = banner_font.render(text, True, COLOR_BANNER)
rect = surf.get_rect(center=area.center)
rect.clamp_ip(surface.get_rect())
surface.blit(surf, rect)
return rect

```

tic-tac-toe/main.py

```

"""
Tic-Tac-Toe (Lab Starter)

Run with: python3 main.py

Click a cell to place your symbol.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE, FONT_NAME, FONT_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Tic-Tac-Toe")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont(FONT_NAME, FONT_SIZE)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)
            elif event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)

        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()

```

ai.py, __init__.py and requirements.txt are unchanged.

Testing

I used pygame 2.6.1 with a dummy display. Clicks went through the real `handle_click` path at pixel coordinates. All of these pass on the new code:

- **All 8 lines:** `check_winner` detects every line for both X and O, and both diagonals end the round when played through clicks.
- **Full-board wins:** a winning last move is reported as an X win, both for a diagonal and for a bottom row. A genuine full-board draw is still reported as "Draw!".
- **Frozen board:** after an X win, an O win, or a draw, clicking all 9 cells changes neither the board nor the turn.

- **R key:** pressing R still starts a fresh round.
- **Task 3 bug kept:** the occupied-cell overwrite still happens mid-round.
- **Random games:** I played 3000 random games. The reported result always matched the board, and the board stayed frozen afterwards.
- **Banner placement:** for every message, the banner rect is inside the window, below the board, and within the margins. Example: (24, 471, 352, 19) for "X wins!".
- **Label:** the turn label reads "Round over" after the round ends.

Running the same checks against the original code fails everything except the Task 3 check. A rendered frame of a diagonal win and of a draw showed the banner fully visible.

One caveat: the container has no Consolas, so pygame fell back to FreeMono. On Windows with real Consolas the text is narrower, so the banner will probably fit at the full size 22. Either way, the shrink-to-fit code covers it.

Exchange 2. Task 3: Turn and move validation

Prompt

Task 1 works. Now Task 3: clicking an occupied cell overwrites it. Reject that click completely so the board and whose turn it is stay exactly the same, and make sure the computer doesn't get a free move from it. Full file please, and test it with clicks on both X and O cells.

Response

The fix

handle_click now returns before touching anything if the clicked cell already holds X or O.

Because the check sits before the board write, the turn swap and `_maybe_take_computer_turn()`, a rejected click changes nothing. That also closes the free-move loophole. Before, clicking your own X "placed" X again, flipped the turn to O, and the computer moved, so the computer got an extra move while you got nothing. Clicking an O cell also stole it from the computer.

Only `game_engine.py` changed. I also removed the `# BUG` comment and the Task 3 note in the docstring, since both are now out of date.

tic-tac-toe/game/game_engine.py

```
"""
GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py) - this is given infrastructure, not
something you need to build.

Starter version: no scoreboard yet, no first-player choice, and only
one combined reset control.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'

class GameEngine:
    def __init__(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = 'X'
        self.round_over = False
        self.winner = None  # 'X', 'O', or None (meaning draw, only valid when round_over)

    def handle_click(self, pos):
        if self.round_over:
            return  # the round has ended - ignore clicks until R starts a new one
        if self.current_player != HUMAN_SYMBOL:
            return  # not your turn - the computer is about to move (or already has)
        cell = board_pos_to_cell(pos)
        if cell is None:
            return
        row, col = cell
        if self.board[row][col] is not None:
            return  # occupied cell - reject the click: no change to the board, the turn, or the computer
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'
        self._maybe_take_computer_turn()
```

```

def _maybe_take_computer_turn(self):
    if self.round_over or self.current_player != COMPUTER_SYMBOL:
        return
    move = choose_move(self.board)
    if move is None:
        return
    row, col = move
    self.board[row][col] = self.current_player
    self.check_round_end()
    self.current_player = 'O' if self.current_player == 'X' else 'X'

def handle_keydown(self, key):
    import pygame
    if key == pygame.K_r:
        self.__init__()

def check_round_end(self):
    winner = check_winner(self.board) # a win takes priority over a full board
    if winner:
        self.round_over = True
        self.winner = winner
    elif is_board_full(self.board):
        self.round_over = True
        self.winner = None

def draw(self, surface, font):
    from game import renderer
    renderer.draw_board(surface, self.board)
    if self.round_over:
        turn_label = "Round over"
    elif self.current_player == HUMAN_SYMBOL:
        turn_label = "Your turn (X)"
    else:
        turn_label = "Computer's turn (O)"
    renderer.draw_text(surface, font, turn_label, (10, 20))

    if self.round_over:
        text = f"{self.winner} wins!" if self.winner else "Draw!"
        renderer.draw_banner(surface, font, f"{text} Press R for a new round.")

```

Testing

All clicks went through the real `handle_click` at pixel coordinates. I wrapped the computer opponent in a counter so the tests could see whether it moved.

- **Click on an X cell:** the board, the turn, `round_over`, `winner` and the computer's move count are all identical before and after.
- **Click on an O cell:** same result, everything unchanged.
- **Cell edges:** clicks on the top-left and bottom-right pixels of an occupied cell are rejected too, not just clicks at the centre.
- **Normal play afterwards:** a valid click after several rejected ones places X and gets exactly one computer reply.
- **Random games:** I played 2000 random games where every turn first clicked every occupied cell, 27,572 rejected clicks on X and O cells in total. None changed any state. X never lost a cell, O never got an extra move, and the reported winner always matched the board.
- **Task 1 still passes:** all the Task 1 tests still pass. The only exception is the one that deliberately checked the overwrite bug was still there, which I removed now that it's fixed.

As a control, I ran the same Task 3 tests on the Task 1 version of the engine. It fails on the first check: clicking your own X changed the state, because the turn flipped and the computer moved.

Exchange 3. Task 2: Persistent scoreboard

Prompt

Now Task 2, the persistent scoreboard. Track X wins, O wins and draws, shown under the turn label as "Score X: n O: n Draw: n". Split the state so `reset_round()` clears only the board (R key, scores kept) and `reset_match()` also clears the scores. Don't bind a key to `reset_match` yet, that's part of Task 4. Each round must be counted exactly once. Full file, and test it over a lot of random rounds.

Response

What changed

The state is split into match state and round state.

- `__init__` now just calls `reset_match()`.
- `reset_match()` zeroes `self.scores` and then calls `reset_round()`.
- `reset_round()` clears only the board, turn, `round_over` and `winner`.
- The R key now calls `reset_round()`. Nothing calls `reset_match()` yet; that's for Task 4.

Each round is counted exactly once.

- Scoring happens in one place, a new `_end_round(winner)` method.
- `check_round_end` returns immediately if `round_over` is already set. So even if it gets called again after the round ends, it can't add a second point.
- Together with the Task 1 click lock, nothing after the end of a round can change the score.

One behaviour choice to be aware of: pressing R in the middle of a round throws that round away without scoring it.

The score line sits 8 px under the turn label, at y = 50, above the board. It is placed from the label's actual drawn position, so it can't overlap the label.

A second file changed: `renderer.py`. A 3-digit score would make the score line wider than the window. So I moved the shrink-to-fit logic from Task 1's banner into a shared `_fit_font` helper. `draw_text` now takes an optional `max_width` and returns its rect. The banner behaves exactly as before. With normal scores the line draws at full size 22; it only shrinks at absurd values like 9999.

`tic-tac-toe/game/game_engine.py`

```
"""
GameEngine: owns the board, turn state, round-end logic and the scoreboard.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py) - this is given infrastructure, not
something you need to build.

State is split in two:
- round state (board, turn, round_over, winner), cleared by reset_round()
- match state (the scoreboard), cleared only by reset_match()

R calls reset_round(). reset_match() is not bound to a key yet.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'
DRAW = 'Draw'
```

```

class GameEngine:
    def __init__(self):
        self.reset_match()

    def reset_match(self):
        self.scores = {'X': 0, 'O': 0, DRAW: 0}
        self.reset_round()

    def reset_round(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = 'X'
        self.round_over = False
        self.winner = None # 'X', 'O', or None (meaning draw, only valid when round_over)

    def handle_click(self, pos):
        if self.round_over:
            return # the round has ended - ignore clicks until R starts a new one
        if self.current_player != HUMAN_SYMBOL:
            return # not your turn - the computer is about to move (or already has)
        cell = board_pos_to_cell(pos)
        if cell is None:
            return
        row, col = cell
        if self.board[row][col] is not None:
            return # occupied cell - reject the click: no change to the board, the turn, or the computer
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'
        self._maybe_take_computer_turn()

    def _maybe_take_computer_turn(self):
        if self.round_over or self.current_player != COMPUTER_SYMBOL:
            return
        move = choose_move(self.board)
        if move is None:
            return
        row, col = move
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'

    def handle_keydown(self, key):
        import pygame
        if key == pygame.K_r:
            self.reset_round()

    def check_round_end(self):
        if self.round_over:
            return # this round has already been scored
        winner = check_winner(self.board) # a win takes priority over a full board
        if winner:
            self._end_round(winner)
        elif is_board_full(self.board):
            self._end_round(None)

    def _end_round(self, winner):
        # The only place a round is scored, and it runs once per round
        # because check_round_end returns early once round_over is set.
        self.round_over = True
        self.winner = winner
        self.scores[winner if winner else DRAW] += 1

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_board(surface, self.board)
        if self.round_over:
            turn_label = "Round over"
        elif self.current_player == HUMAN_SYMBOL:
            turn_label = "Your turn (X)"
        else:
            turn_label = "Computer's turn (O)"
        label_rect = renderer.draw_text(surface, font, turn_label, (renderer.TEXT_MARGIN, 20))

```

```

score_text = f"Score X: {self.scores['X']} O: {self.scores['O']} Draw: {self.scores[DRAW]}"
renderer.draw_text(
    surface, font, score_text,
    (renderer.TEXT_MARGIN, label_rect.bottom + 8),
    max_width=surface.get_width() - 2 * renderer.TEXT_MARGIN,
)

if self.round_over:
    text = f"{self.winner} wins!" if self.winner else "Draw!"
    renderer.draw_banner(surface, font, f"{text} Press R for a new round.")

```

tic-tac-toe/game/renderer.py

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import functools

import pygame

WIDTH, HEIGHT = 400, 500
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
TEXT_MARGIN = 10
WINDOW_SIZE = (WIDTH, HEIGHT)

FONT_NAME = "consolas"
FONT_SIZE = 22
MIN_FONT_SIZE = 12

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
COLOR_BANNER = (180, 40, 40)

@functools.lru_cache(maxsize=None)
def _font(size):
    return pygame.font.SysFont(FONT_NAME, size)

def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP
    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None
    col = x // CELL_SIZE
    row = y // CELL_SIZE
    return int(row), int(col)

def draw_board(surface, board):
    surface.fill(COLOR_BG)
    for i in range(1, 3):
        pygame.draw.line(surface, COLOR_LINE, (i * CELL_SIZE, BOARD_TOP), (i * CELL_SIZE, BOARD_TOP + BOARD_SIZE), 3)
        pygame.draw.line(surface, COLOR_LINE, (0, BOARD_TOP + i * CELL_SIZE), (BOARD_SIZE, BOARD_TOP + i * CELL_SIZE), 3)

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]
            if symbol is None:
                continue
            center = (c * CELL_SIZE + CELL_SIZE // 2, BOARD_TOP + r * CELL_SIZE + CELL_SIZE // 2)

```

```

        if symbol == 'X':
            offset = CELL_SIZE // 3
            pygame.draw.line(surface, COLOR_X, (center[0]-offset, center[1]-offset), (center[0]+offset,
center[1]+offset), 6)
            pygame.draw.line(surface, COLOR_X, (center[0]+offset, center[1]-offset), (center[0]-offset,
center[1]+offset), 6)
        else:
            pygame.draw.circle(surface, COLOR_O, center, CELL_SIZE // 3, 6)

def _fit_font(font, text, max_width, max_height=None):
    # Return font if text fits; otherwise the largest smaller size that does.
    fitted = font
    width, height = fitted.size(text)
    size = FONT_SIZE
    while (width > max_width or (max_height is not None and height > max_height)) and size > MIN_FONT_SIZE:
        size -= 1
        fitted = _font(size)
        width, height = fitted.size(text)
    return fitted

def draw_text(surface, font, text, pos, color=COLOR_TEXT, max_width=None):
    if max_width is not None:
        font = _fit_font(font, text, max_width)
    surf = font.render(text, True, color)
    rect = surf.get_rect(topleft=pos)
    surface.blit(surf, rect)
    return rect

def draw_banner(surface, font, text):
    # The banner lives in the strip between the bottom of the board and
    # the bottom of the window, with a small margin on the left and right.
    strip_top = BOARD_TOP + BOARD_SIZE
    area = pygame.Rect(
        TEXT_MARGIN,
        strip_top,
        surface.get_width() - 2 * TEXT_MARGIN,
        surface.get_height() - strip_top,
    )

    # Shrink the font if needed so the banner is never cut off at the window edges.
    banner_font = _fit_font(font, text, area.width, area.height)

    surf = banner_font.render(text, True, COLOR_BANNER)
    rect = surf.get_rect(center=area.center)
    rect.clamp_ip(surface.get_rect())
    surface.blit(surf, rect)
    return rect

```

rules.py and main.py are the same as the Task 1 versions.

Testing

20,000 random rounds in one match. They were played through real pixel clicks, with about 30% extra clicks on occupied cells mixed in. After every round:

- The score had to equal an independent tally computed from the final board. The final tally was X 11,693, O 5,832, Draw 2,475, summing to exactly 20,000.
- I then tried to double count: called `check_round_end()` 3 more times, drew 3 more frames, and clicked all 9 cells. The score never moved.
- R cleared the round (empty board, X to move, no winner) and left the scores untouched.
- Every 97th round I also pressed M and Space; neither one changed the board or the score.

Other checks:

- A new engine starts at 0/0/0.

- R pressed mid-round clears the board without scoring.
- `reset_match()` clears both the scores and the board, whether called mid-round or after a round has ended.

Rendering:

- The text matches your format exactly.
- It sits below the turn label, above the board, and inside the 10 px margins. Examples: (10, 50, 351, 22) for 12/7/3, and a shrunk (10, 50, 350, 17) for 9999/9999/9999.
- A rendered frame after 5 rounds shows "Score X: 4 O: 0 Draw: 1" under "Round over".

Regression: all the Task 1 and Task 3 tests still pass, and `main.py` starts and runs without crashing.

Two of my old Task 1 test helpers needed small updates, because `draw_text` now returns a rect and is called twice per frame. Neither change was a game bug.

Exchange 4. Task 4: First-player choice and separate restart controls

Prompt

Last one, Task 4. Add first-player choice: X and O keys set who starts the next round (not the current one), and if O starts, the computer moves immediately. R restarts the round keeping the score, M resets the match and clears it, and the starter choice should survive M. Make the window taller and show these controls in small text below the board, and check everything fits in the 400px width. Update the main.py docstring too. Full files and test it.

Response

Task 4 is in, and all four tasks' tests pass together.

What changed

The starter choice is its own piece of state. The engine now has `next_starter`. It is set to X once in `__init__`, and only the X and O keys change it. `reset_round()` and `reset_match()` never touch it, so the choice survives both R and M.

X and O only affect the next round. Pressing them only updates `next_starter`. The board and turn of the current round stay exactly as they were.

If O starts, the computer moves right away. `reset_round()` sets `current_player` from `next_starter` and then calls `_maybe_take_computer_turn()`. So when O is chosen, the computer makes its opening move the moment R or M starts the round, and the screen already shows "Your turn (X)".

R and M are separate. R calls `reset_round()` and keeps the score. M calls `reset_match()`, which clears the score.

The layout is now built from named regions. The window is 400 × 580: label and score, then the board, then the 40 px banner strip, then an 80 px controls area. The banner is centred in its own strip instead of "everything below the board".

The controls text sits below a thin divider. It's three lines of grey size-16 text. The last line shows the pending choice ("next: O"), so the X/O key press is visible on screen even though the current round doesn't change. Each line uses the same shrink-to-fit logic as the score line and banner.

The main.py docstring now lists all the controls. Nothing else in `main.py` changed; the window size comes from `WINDOW_SIZE`.

tic-tac-toe/game/game_engine.py

```
"""
GameEngine: owns the board, turn state, round-end logic and the scoreboard.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py) - this is given infrastructure, not
something you need to build.

State is split in three:
- round state (board, turn, round_over, winner), cleared by reset_round()
- match state (the scoreboard), cleared by reset_match()
- the starter choice (next_starter), only changed by the X and O keys,
  so it survives both R and M

Keys: R restarts the round, M resets the match, X / O choose who starts
the next round. If O starts, the computer moves as soon as the round begins.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
```

```

from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'
DRAW = 'Draw'

class GameEngine:
    def __init__(self):
        self.next_starter = HUMAN_SYMBOL
        self.reset_match()

    def reset_match(self):
        self.scores = {'X': 0, 'O': 0, DRAW: 0}
        self.reset_round()

    def reset_round(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = self.next_starter
        self.round_over = False
        self.winner = None # 'X', 'O', or None (meaning draw, only valid when round_over)
        self._maybe_take_computer_turn() # if O starts, the computer opens the round

    def handle_click(self, pos):
        if self.round_over:
            return # the round has ended - ignore clicks until R starts a new one
        if self.current_player != HUMAN_SYMBOL:
            return # not your turn - the computer is about to move (or already has)
        cell = board_pos_to_cell(pos)
        if cell is None:
            return
        row, col = cell
        if self.board[row][col] is not None:
            return # occupied cell - reject the click: no change to the board, the turn, or the computer
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'
        self._maybe_take_computer_turn()

    def _maybe_take_computer_turn(self):
        if self.round_over or self.current_player != COMPUTER_SYMBOL:
            return
        move = choose_move(self.board)
        if move is None:
            return
        row, col = move
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'

    def handle_keydown(self, key):
        import pygame
        if key == pygame.K_r:
            self.reset_round()
        elif key == pygame.K_m:
            self.reset_match()
        elif key == pygame.K_x:
            self.next_starter = HUMAN_SYMBOL # takes effect from the next round
        elif key == pygame.K_o:
            self.next_starter = COMPUTER_SYMBOL # takes effect from the next round

    def check_round_end(self):
        if self.round_over:
            return # this round has already been scored
        winner = check_winner(self.board) # a win takes priority over a full board
        if winner:
            self._end_round(winner)
        elif is_board_full(self.board):
            self._end_round(None)

    def _end_round(self, winner):
        # The only place a round is scored, and it runs once per round
        # because check_round_end returns early once round_over is set.
        self.round_over = True

```

```

        self.winner = winner
        self.scores[winner if winner else DRAW] += 1

    def controls_lines(self):
        return [
            "R restart round, keep score",
            "M reset match, clear score",
            f"X / O pick next starter (next: {self.next_starter})",
        ]

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_board(surface, self.board)
        if self.round_over:
            turn_label = "Round over"
        elif self.current_player == HUMAN_SYMBOL:
            turn_label = "Your turn (X)"
        else:
            turn_label = "Computer's turn (O)"
        label_rect = renderer.draw_text(surface, font, turn_label, (renderer.TEXT_MARGIN, 20))

        score_text = f"Score X: {self.scores['X']} O: {self.scores['O']} Draw: {self.scores[DRAW]}"
        renderer.draw_text(
            surface, font, score_text,
            (renderer.TEXT_MARGIN, label_rect.bottom + 8),
            max_width=surface.get_width() - 2 * renderer.TEXT_MARGIN,
        )

        if self.round_over:
            text = f"{self.winner} wins!" if self.winner else "Draw!"
            renderer.draw_banner(surface, font, f"{text} Press R for a new round.")

        renderer.draw_controls(surface, self.controls_lines())

```

tic-tac-toe/game/renderer.py

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import functools

import pygame

# Vertical layout, top to bottom: turn label and score (0-100), board,
# result banner strip, then the controls help text.
WIDTH = 400
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
BANNER_TOP = BOARD_TOP + BOARD_SIZE
BANNER_HEIGHT = 40
CONTROLS_TOP = BANNER_TOP + BANNER_HEIGHT
CONTROLS_HEIGHT = 80
HEIGHT = CONTROLS_TOP + CONTROLS_HEIGHT
WINDOW_SIZE = (WIDTH, HEIGHT)
TEXT_MARGIN = 10

FONT_NAME = "consolas"
FONT_SIZE = 22
SMALL_FONT_SIZE = 16
MIN_FONT_SIZE = 10
LINE_GAP = 4

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
COLOR_BANNER = (180, 40, 40)
COLOR_HELP = (100, 100, 100)
COLOR_DIVIDER = (200, 200, 200)

```

```

@functools.lru_cache(maxsize=None)
def _font(size):
    return pygame.font.SysFont(FONT_NAME, size)

def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP
    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None
    col = x // CELL_SIZE
    row = y // CELL_SIZE
    return int(row), int(col)

def draw_board(surface, board):
    surface.fill(COLOR_BG)
    for i in range(1, 3):
        pygame.draw.line(surface, COLOR_LINE, (i * CELL_SIZE, BOARD_TOP), (i * CELL_SIZE, BOARD_TOP +
BOARD_SIZE), 3)
        pygame.draw.line(surface, COLOR_LINE, (0, BOARD_TOP + i * CELL_SIZE), (BOARD_SIZE, BOARD_TOP + i *
CELL_SIZE), 3)

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]
            if symbol is None:
                continue
            center = (c * CELL_SIZE + CELL_SIZE // 2, BOARD_TOP + r * CELL_SIZE + CELL_SIZE // 2)
            if symbol == 'X':
                offset = CELL_SIZE // 3
                pygame.draw.line(surface, COLOR_X, (center[0]-offset, center[1]-offset), (center[0]+offset,
center[1]+offset), 6)
                pygame.draw.line(surface, COLOR_X, (center[0]+offset, center[1]-offset), (center[0]-offset,
center[1]+offset), 6)
            else:
                pygame.draw.circle(surface, COLOR_O, center, CELL_SIZE // 3, 6)

def _fit_font(font, text, max_width, max_height=None, start_size=FONT_SIZE):
    # Return font if text fits; otherwise the largest smaller size that does.
    fitted = font
    width, height = fitted.size(text)
    size = start_size
    while (width > max_width or (max_height is not None and height > max_height)) and size > MIN_FONT_SIZE:
        size -= 1
        fitted = _font(size)
        width, height = fitted.size(text)
    return fitted

def draw_text(surface, font, text, pos, color=COLOR_TEXT, max_width=None):
    if max_width is not None:
        font = _fit_font(font, text, max_width)
    surf = font.render(text, True, color)
    rect = surf.get_rect(topleft=pos)
    surface.blit(surf, rect)
    return rect

def draw_banner(surface, font, text):
    # The banner lives in the strip between the bottom of the board and
    # the controls text, with a small margin on the left and right.
    area = pygame.Rect(
        TEXT_MARGIN,
        BANNER_TOP,
        surface.get_width() - 2 * TEXT_MARGIN,
        BANNER_HEIGHT,
    )

```

```

# Shrink the font if needed so the banner is never cut off at the window edges.
banner_font = _fit_font(font, text, area.width, area.height)

surf = banner_font.render(text, True, COLOR_BANNER)
rect = surf.get_rect(center=area.center)
rect.clamp_ip(surface.get_rect())
surface.blit(surf, rect)
return rect

def draw_controls(surface, lines):
    # Small grey help text under the banner strip, one control per line.
    pygame.draw.line(surface, COLOR_DIVIDER, (TEXT_MARGIN, CONTROLS_TOP), (surface.get_width() - TEXT_MARGIN,
CONTROLS_TOP), 1)
    max_width = surface.get_width() - 2 * TEXT_MARGIN
    small = _font(SMALL_FONT_SIZE)
    y = CONTROLS_TOP + 8
    rects = []
    for line in lines:
        line_font = _fit_font(small, line, max_width, start_size=SMALL_FONT_SIZE)
        surf = line_font.render(line, True, COLOR_HELP)
        rect = surf.get_rect(topleft=(TEXT_MARGIN, y))
        surface.blit(surf, rect)
        rects.append(rect)
        y = rect.bottom + LINE_GAP
    return rects

```

tic-tac-toe/main.py

```

"""
Tic-Tac-Toe against a random-move computer opponent.

Run with: python3 main.py

You play X, the computer plays O and moves automatically after you.

Controls:
Click a cell    place your X (occupied cells and finished rounds ignore clicks)
R              restart the round, keeping the score
M              reset the match, clearing the score (the starter choice is kept)
X / O          choose who starts the next round; if O starts, the computer
                makes the first move as soon as the round begins
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE, FONT_NAME, FONT_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Tic-Tac-Toe")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont(FONT_NAME, FONT_SIZE)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)
            elif event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)

        engine.draw(screen, font)

    pygame.display.flip()

```

```

        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()

```

rules.py, ai.py, __init__.py and requirements.txt are unchanged from the earlier tasks or the starter.

Testing

Specific scenarios. All of these pass:

- A new game starts with X to move, an empty board, and no computer move.
- Pressing O, X, O in the middle of a round changes only next_starter. The board, turn and score stay the same, and the computer makes no move.
- R with O chosen: the computer opens with exactly one move, and then it's your turn.
- R with X chosen: empty board, your turn.
- Choosing a starter after a round has finished leaves the finished board alone and applies to the next round.
- M clears the score and the round, and the starter choice survives five M and R presses in a row.
- R keeps the score across rounds.

Long random run. I sent 200,000 random clicks and key presses, mixing R, M, X, O and an unbound Q. After every single step the test checked that:

- next_starter matched what the keys had set.
- The X and O counts on the board matched whoever started that round: the starter has the same count or one more.
- Mid-round, it was always your turn after any action.
- The score matched an independent tally. Across the run, 4,892 rounds finished and each was scored exactly once.

O-start rounds. Over 3,000 rounds where O starts, all three outcomes happened: X wins, O wins and draws. This matters because when O starts, the computer is the one that fills the last cell.

Layout. I drew every combination of: starter X or O, scores of 0, 99 or 9999, and in play, X won, O won or draw. In every case all text stayed within the 10 px margins of the 400 px width. The widest right edge was 387 px against a limit of 390 px. The score line sits above the board, the banner stays inside its strip, the controls lines don't overlap each other, and everything ends by y = 580.

Rendered frames showed:

- an O-start round where the computer has already played and the screen says "Your turn (X)";
- a finished round with the score, the banner and the controls text all visible.

Regressions. The Task 1, 2 and 3 suites all still pass. In the Task 2 test I swapped M for Q as the example of a key that does nothing, since M now has a job. main.py starts and runs without crashing.

For your "after" video, a good sequence:

1. Win a diagonal.
2. Press R and show the score is kept.
3. Press O, then R, and show the computer moves first.
4. Click an occupied cell to show it's rejected.
5. Press M and show the score clears while the controls still say "next: O".